



南京大學

本科畢業論文

院 系 智能軟件工程學院

專 業 軟件工程

題 目 基於 k-kick 樹的最優時空權衡

哈希表的設計與實現

年 級 2022 學 號 221900387

學生姓名 肖揚

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 5 月 12 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于 k-kick 树的最优时空权衡哈希表的设计与实现

院系：智能软件工程学院

专业：软件工程

本科生姓名：肖扬

指导教师（姓名、职称）：刘明谋 副教授

摘要：

哈希表是支持动态数据操作的基础数据结构，其时空权衡一直是算法研究的核心议题。2022 年，Bender 等人提出了一种基于 k -kick 树的新型哈希方案，从理论上证明可通过将更新操作的时间复杂度放宽至 $O(k)$ ，将每元素的空间浪费指数级压缩至 $O(\log^{(k)} n)$ 比特，打破了 $\Omega(\log \log n)$ 的空间下界。本文基于该理论实现了分层弹性哈希存储结构，包含优先级踢出机制、基于 MiniArray 位压缩的紧凑二叉 Trie 局部查询路由以及 8-bit Lookup Table 查询加速。槽元数据和空闲槽位图分别通过双层 MiniArray 压缩至每槽 4 比特和 1 比特，Trie 节点以紧凑位编码存储。在 $n \leq 10^6$ 个 32 位键值对的实验规模下，与 `std::unordered_map` 进行了插入、查询、删除和空间占用的对比。实验表明：插入在 $n \leq 2 \times 10^5$ 时每操作仅 ~ 0.8 us，合并策略避免了 $O(N)$ 全量路由重建；查询在 LUT 加速下达 ~ 1.95 us/op；空间占用在 $n = 2 \times 10^5$ 时约为 9 MB，在 $n = 10^6$ 时为 97 MB，通过动态 cubby 容量控制合并空间效率。最后将方案集成为 LevelDB MemTable 的点查询加速器，提供编译期和运行时双向开关，集成实验验证适配层开销仅 5.17 ns/次，查询性能在 $n = 2 \times 10^5$ 时达 0.35 us/op。

关键词：哈希表，时空权衡，k-kick 树，位压缩路由，LevelDB

关键词：哈希表；时空权衡；k-kick 树；位压缩路由；LevelDB

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design and Implementation of Optimal Time/Space Tradeoff Hash Table

Based on k-kick Trees

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software and Engineering

UNDERGRADUATE: Yang Xiao

MENTOR: Associate Professor Mingmou Liu

ABSTRACT:

Hash tables are fundamental data structures supporting dynamic data operations, with their time-space tradeoffs being a core topic in algorithm research. In 2022, Bender et al. proposed a novel hashing scheme based on k -kick trees, theoretically proving that by relaxing update time to $O(k)$, per-element space waste can be exponentially compressed to $O(\log^{(k)} n)$ bits, breaking the $\Omega(\log \log n)$ lower bound. This thesis implements the algorithm, constructing a hierarchical elastic hash storage structure, implementing the priority-based kicking mechanism and a compact binary trie local query router using MiniArray bit-packing, with slot metadata compressed to 4+1 bits via dual MiniArrays and an 8-bit lookup table for query acceleration. Experiments with $n \leq 10^6$ 32-bit key-value pairs compare insertion, query, deletion and space against `std::unordered_map`. The implementation achieves ~ 0.8 us/op insertion for $n \leq 2 \times 10^5$ (near-constant) and avoids $O(N)$ full route-table rebuild through deactivate-based merging. Query time is ~ 1.95 us/op at $n = 10^6$ with LUT acceleration. Space is 9 MB at $n = 2 \times 10^5$ (45 B/key) and 97 MB at $n = 10^6$, controlled by dynamic cubby sizing. The scheme is integrated into LevelDB's MemTable with a `-DUSE_KICK_HASH` compile-time switch and `SetUseKickHash()` runtime toggle. Integration benchmarks show adapter overhead of only 5.17 ns/op and query performance of 0.35 us/op at $n = 2 \times 10^5$.

Keywords: hash table, time-space tradeoff, k-kick tree, bit-packed routing, LevelDB

KEYWORDS: hash table; time-space tradeoff; k-kick tree; elastic hashing; bit-packed routing; LevelDB

目 录

第一章 引言

1.1 研究背景和意义

哈希表 (Hash Table) 是现代数据库管理系统、分布式缓存系统和全文搜索引擎中最为基础的数据结构之一。从早期的内存键值存储到如今的大规模分布式 NoSQL 系统, 高效的数据索引和查找始终是决定系统整体吞吐量的关键瓶颈。哈希表的理论目标十分明确: 支持动态插入和删除操作, 并保证查询的时间复杂度为 $O(1)$, 即在理想状态下, 三种基本操作所需时间均为常数级, 不随存储数据量 n 的增大而增加。然而, 在实际系统设计中, 为了保证这一理想时间复杂度, 哈希表不得不在空间占用方面做出让步——通过分配额外的冗余比特位来降低冲突概率并维持探测效率。

近年来, 随着互联网应用数据量的爆炸式增长, 键值存储 (Key-Value Store) 系统在工业界和学术界均受到了广泛关注。以 Google LevelDB²、Facebook RocksDB 和 Apache Cassandra 为代表的 LSM 树 (Log-Structured Merge-Tree) 存储引擎已在大规模生产环境中得到部署, 国内学者围绕 LSM 树的写入放大、读性能优化和压缩策略等方面开展了大量研究工作¹³。张伟等人的综述指出, LSM 树的核心瓶颈在于多层合并带来的写放大问题, 而 MemTable 层的索引效率直接决定了写入路径的吞吐上限, 因此寻找更高效的内存索引结构成为改进 LSM 树系统的关键方向之一¹³。此外, 刘洋和陈静的研究¹⁴针对 LevelDB 中 MemTable 的 SkipList 索引进行了混合存储优化的探索, 尝试在 SkipList 基础上引入缓存友好型辅助索引来加速点查询操作。这些工作表明, 优化 MemTable 层的底层数据结构对提升数据库整体性能具有显著意义, 但现有方案在空间效率方面仍存在较大的改进空间。

在理论层面, 要求插入、删除及查询三种操作均维持恒定时间复杂度 (即期望时间或最坏情况时间为 $O(1)$) 的前提下, 早期的理论工作^{3,10}表明, 在严格限制更新操作为 $O(1)$ 时间且仅使用简单探测序列的模型下, 每个键值对的空间冗

余往往难以低于 $\Omega(\log \log n)$ 比特。1984 年, Fredman、Komlós 和 Szemerédi 提出的 FKS 完美哈希方案¹⁰首次在严格 $O(1)$ 最坏查询时间下实现了线性总空间, 为哈希表时空权衡研究奠定了理论基础。然而在此后近四十年的时间里, 上述 $\Omega(\log \log n)$ 的冗余下界始终未被突破, 长期制约着哈希表空间效率的进一步提升。

2022 年, Bender 等人发表的论文《关于哈希表最优时间/空间权衡》¹ 打破了这一理论僵局。该研究提出了一种基于 k -kick 树的全新哈希方案, 从理论上严格证明了通过适当放宽插入和删除操作的时间复杂度约束——从经典的 $O(1)$ 提高到 $O(k)$ (其中 k 是一个可由用户指定的、具有上界的常数)——可以将每元素的空间浪费从 $\Omega(\log \log n)$ 比特以指数级幅度压缩至 $O(\log^{(k)} n)$ 比特。这里 $\log^{(k)} n$ 表示对 n 进行 k 次迭代对数运算, 即递归定义 $\log^{(k)} n = \log(\log^{(k-1)} n)$, 并规定 $\log^{(0)} n = n$ 、 $\log^{(1)} n = \log n$ 。特别地, 当 $k = \log^* n$ (迭代对数函数, 定义为对 n 连续取对数直到结果收敛至不大于 1 所需的次数) 时, 空间浪费可以进一步降低至恒定的 $O(1)$ 比特水平, 在理论和工程两个层面均构成了重大突破。

从工程应用的角度来看, 内存占用对数据库系统的实际运行成本具有直接影响。赵宏等人的研究¹⁵指出, 在内存受限的嵌入式数据库和边缘计算场景中, 哈希索引结构的每字节空间开销都会显著影响系统的可部署性和扩展能力。通过降低哈希表的冗余比特数, 可以在不增加硬件投入的前提下支撑更大的数据规模, 从而降低单位数据的存储成本和能耗开销。与此同时, 内存效率的提升意味着在同样的物理服务器上可以部署更多的数据库实例, 间接提高了系统的并发处理能力和资源利用率。因此, 本研究在理论意义之外同样具有现实的工程应用价值。

本毕业设计旨在系统性地复现 Bender 等人提出的 k -kick 树哈希算法, 并在代码实现的基础上将其适配集成至 Google LevelDB 这一成熟的开源存储引擎之中。研究意义可以从理论和工程两个维度加以阐述: 从理论层面来看, 通过可控的实验验证论文所断言的指数级空间压缩效应, 并分析参数 k 的取值对时间与空间性能的具体量化影响; 从工程层面来看, 将这一前沿理论成果落地转化为可实际运行的数据库组件, 观察和分析其在真实存储系统中的表现, 有助于弥合算法理论与系统实现之间的鸿沟。

1.2 研究现状

哈希表作为支持近乎常数时间的插入、删除和查询操作的核心数据结构，其时间复杂度和空间复杂度之间的权衡关系始终是现代算法理论与系统实现领域的研究焦点。在国内外的数据库研究社区中，如何设计时间高效且空间紧凑的哈希索引始终是存储引擎优化的核心议题之一¹⁸。孙建国等人在综述中指出，当前内存键值存储系统中的哈希索引方案大致可划分为基于链式寻址、基于开放寻址、基于 Cuckoo Hashing 及其变体、以及基于紧凑编码四个主要技术路线¹⁸，不同路线在空间效率、查询延迟和写入吞吐之间分别做出了各自的权衡取舍。

1.2.1 链式寻址法

链式寻址法（Separate Chaining）是最为直观的冲突解决策略，其基本思路是为哈希表中的每个槽维护一个独立的链表数据结构，所有被映射到同一槽位的键值对均以链表节点的形式链接存储。该方法的优点在于实现逻辑简洁清晰，且能够比较方便地支持动态扩容操作——当负载因子超过预设阈值时，只需分配更大的槽数组并将所有节点重新插入即可。然而，链式寻址法也存在不容忽视的固有缺陷：首先，每个链表节点都需要额外存储至少一个后继指针，这在高负载因子场景下会造成显著的空间开销；其次，链表节点在堆内存中的非连续分配模式严重破坏了 CPU 缓存的局部性（cache locality），在遍历长冲突链时极易触发缓存缺失（cache miss），进而导致查询延迟的显著增加。在数据量较大的实际应用场景中，频繁的指针追踪和不可预测的内存访问模式往往成为制约系统吞吐量的主要瓶颈。

1.2.2 开放寻址法

开放寻址法（Open Addressing）放弃了链表的辅助存储，转而在一个连续的槽数组内部直接解决冲突。其核心操作流程是：当待插入元素的哈希值对应的目标槽位已被其他元素占据时，按照一个预先定义的、确定的探测序列（probe sequence）在槽数组中继续搜索下一个可用位置。典型的探测策略包括线性探测（linear probing）、二次探测（quadratic probing）和双重哈希（double hashing）。开放寻址法的显著优势在于所有元素连续存储在单一数组之中，内存访问具有高

度的空间局部性，在负载因子适中的情况下可以实现极佳的缓存性能。然而，随着负载因子 α 逐渐趋近于 1，探测序列的长度会呈现急剧增长的趋势，导致查询和插入操作在长序列上的时间开销迅速退化。正因如此，开放寻址法在实际使用中通常需要将负载因子严格控制在较低水平——例如 0.5 至 0.7 之间——以避免性能的雪崩式下降。在严格的负载限制下，开放寻址法尽管在时间效率方面表现优秀，但在空间利用率维度上存在天然的局限。

1.2.3 Cuckoo Hashing

Cuckoo Hashing 由 Pagh 和 Rodler 于 2001 年首次提出⁴，其设计思想与传统的开放寻址和链式寻址有着本质区别。该方案通过引入多候选位置策略及元素重定位（relocation）机制，在最坏情况下实现了严格 $O(1)$ 的查询时间上界。与单一探测序列函数逐次生成探测位置的做法不同，Cuckoo Hashing 转而采用多个独立哈希函数，为每个待存储的元素同时生成若干个互不相关的可存入位置，硬件实现时可以并行检查这些位置。更进一步地，其重定位机制在检测到冲突时不再被动退让，而是主动将某个已占位置的现有元素“踢出”（kick out），并将被踢元素重新安置到属于它自身的其他候选位置之中。这两位一体的机制协同配合，严格保证了任意键的查找最多只需计算并检查预设的哈希函数个数个位置，从机制上确保了常数级查询时间。

黄磊和杨帆¹⁷针对 Cuckoo Hashing 方案在实际键值存储引擎中的落地方案进行了系统性研究，指出经典 Cuckoo Hashing 在负载因子接近 50% 时踢出循环的概率急剧增加，会导致单次插入的延迟出现较为显著的抖动（jitter）。他们提出通过预留少量缓存槽（stash）来吸收溢出元素，在不破坏查询语义的前提下将插入的摊销代价维持在可接受范围内。这一研究揭示了 Cuckoo Hashing 方案从理论到工程落地之间需要克服的若干实际问题，包括踢出循环检测、多线程并发控制和缓存槽的溢出处理等。

Arbitman、Naor 和 Segev⁸进一步提出了去摊还（de-amortized）Cuckoo Hashing 方案，通过队列化踢出操作缓冲区实现了确定性的最坏情况性能保证。该方案利用全局预取技术来预先分配踢出链可能用到的空闲槽位，从而将经典方案中依赖高概率分析的踢出深度转化为确定性的、可控的 $O(1)$ 边界。这一改进使得 Cuckoo Hashing 的工程实用性得到了显著提升。

1.2.4 基于 k-kick 树的时空最优方案

前述的链式寻址、开放寻址和 Cuckoo Hashing 及其衍生方案在时间效率方面均已取得了长足进展，但无一例外地未能突破 $\Omega(\log \log n)$ 比特/键的空间冗余下界。2022 年，Bender 等人发表的《关于哈希表时间/空间权衡的最优方案》¹中提出的 k -kick 树哈希方案是目前已知的第一个在理论上严格突破该下界的成果。该方案的核心思想是将哈希表的键-槽分配问题转化为一个层次化的“球进桶”（balls-into-bins）过程：通过引入随机分配的最大深度优先级和受限于参数 k 的有限次元素移动（即“ k -kick”机制），从理论上证明了对于任何预先选定的 $k \geq 1$ ，可以同时实现 $O(k)$ 时间复杂度的元素更新（覆盖插入、删除和查询三种操作）和 $O(\log^{(k)} n)$ 比特的每元素空间浪费，从而建立了一个完整的、可连续调节的时空权衡曲线。

该研究还进一步证明了上述权衡曲线对所有基于“增强开放寻址”（augmented open addressing）框架的哈希表方案而言是紧确（tight）的¹。具体而言，任何遵循增强开放寻址范式的方案若限定了其更新时间复杂度为 t ，则每个元素至少需要使用 $\Omega(\log^{(t)} n)$ 比特的空间冗余。这一紧确性结论意味着，若研究者希望在次对数级别的更新时间下将空间冗余进一步压缩至 $O(1)$ 比特水平，必须从根本上放弃增强开放寻址的框架约束，转而寻求全新的数据结构设计范式。这一发现对哈希表的理论研究方向具有深远的指引意义。

1.2.5 紧凑字典与空间高效结构

在哈希表空间效率的研究方向上，还有一条并行推进的技术路线值得关注。Raman 和 Rao¹²提出了紧凑动态字典（succinct dynamic dictionaries）和相应的紧凑树形存储结构，通过递归式的位级编码策略，在维持 $O(1)$ 最坏查询时间的同时将整体空间占用压缩至与信息论下界相差仅常数倍的水平，为动态哈希表的紧凑化存储提供了核心的技术参考。Liu、Yin 和 Yu¹¹则进一步将紧凑化的思想拓展到过滤器（filter）领域，研究了对未知大小集合进行紧凑编码的自适应空间分配策略，在动态场景下实现了与静态结构相当的压缩效率。

上述工作在“紧致性”（空间趋近理论下界）与“动态性”（支持高效的在线插入和删除）之间分别采取了不同的权衡策略。而 Bender 等人的 k -kick 树方案¹的

独特贡献在于：它首次给出了一个完整的、从 $k = 1$ 到 $k = \log^* n$ 可连续调节的时空权衡理论曲线，揭示了每次以常数因子增加更新时间所能换取的空间效率提升是以指数级的幅度递增的。这一理论结果为后续的哈希表设计提供了基准性的参考框架。

1.3 主要研究内容和目标

本毕业设计选题的核心任务在于完整复现并实验验证 Bender 等人于 2022 年提出的 k -kick 树哈希算法¹，并以 Google LevelDB²为目标平台，探索该算法在真实数据库存储引擎中的集成可行性与性能表现。具体而言，研究工作围绕以下四个递进的目标层次展开：

1. **核心数据结构的实现与算法复现。**严格依据原始论文¹中公示的数据结构定义和算法伪代码，以 C++20 标准构建基于 k -kick 树的完整四层分层存储系统。实现方面包括：MiniArray 位压缩数组原语的双模存储及其位级操作接口的精确实现、LocalQueryRouter 紧凑二叉 Trie 查询路由的构建与维护算法（覆盖 walk-down 查询、do_split 分裂插入和标记式删除）、Cubby 弹性哈希表中优先级踢出置换链的递归搜索与恢复机制，以及 Facility 全局分层管理器中基于 deactivate 语义的合并与路由更新策略。各组件均需在代码层面与论文描述的设计意图保持一致。
2. **适配层的构建与数据库存储集成。**在完整实现核心算法的基础上，设计并实现面向 LevelDB MemTable 的适配接口层。任务包括：基于 Slice 到 64-bit 哈希的类型映射转换、KickHashTable 包装器类的封装、MemTable 写入路径（Add）和读取路径（Get）的条件编译改造，以及编译期（CMake 选项）与运行时（C++ 成员函数）双层级开关机制的研发。适配层需要在不破坏 LevelDB 原有代码结构和异常处理策略的前提下，完成两组代码路径的融合。
3. **理论正确性的代码级验证与时空复杂度分析。**基于已完成的 C++ 实现，通过构造覆盖边界条件的测试用例集，逐项验证实实现与理论模型之间的一致性。具体验证维度包括：踢出链长度在任意插入序列下不超过 k 的理论上界、两级 Trie 路由的指纹碰撞率是否符合随机二元串模型的理论预期、以

及空间浪费 $S(n)$ 随数据规模 n 的变化趋势是否满足 $O(\log^{(k)} n)$ 的理论渐近规律。

4. **全面的性能对比实验与集成效果评估。**以 C++ 标准库中的 `std::unordered_map` 作为基准对照组，在涵盖从 10^3 到 10^6 九个数量级的数据规模下，系统地测量并对比插入、查询和删除三种操作的时间开销以及进程的内存占用情况。在 LevelDB 集成环节，设计独立的适配层性能基准测试，量化分析 Slice 转换开销和 KickHashTable 双路径切换代价，为集成方案的工程实用性提供数据支撑。

1.4 论文结构

本文共分为七个章节展开论述。第一章为引言部分，系统性地介绍研究背景、国内外研究现状、本文的研究目标以及全文的组织结构。第二章围绕相关理论基础展开，涵盖哈希表的基本概念与操作模型、哈希函数的设计准则与通用哈希族的理论基础、Cuckoo Hashing 的核心机制与去摊还改进，以及 k -kick 树哈希方案的完整理论框架，包括多层级 bin 划分、优先级踢出策略和渐进复杂度保证。第三章详细阐述系统的设计方案，按照从底层原语到顶层管理器的自底向上顺序，逐一介绍 MiniArray、LocalQueryRouter、Cubby 和 Facility 四个组件的设计动机、接口定义与内部机理。第四章聚焦于 C++ 实现层面的具体细节，包括各组件中关键函数的算法流程、代码结构组织以及整体复杂度的汇总分析。第五章呈现完整的实验评估结果：在基准硬软件环境下，对多规模数据在插入、查询、删除和内存四种维度上对 k -kick 方案与标准库对比对象进行定量分析。第六章专门描述 k -kick 方案与 LevelDB 数据库²的集成工作，涵盖适配层设计、MemTable 改造、双向开关机制和集成效果验证实验。第七章对全文工作进行全面总结，客观分析当前方案的已有成果与尚存不足，并对未来的改进方向和可能的后续研究方向进行展望。

第二章 相关理论基础

2.1 哈希表基本概念

哈希表 (Hash Table) 是一种通过单射或近单射的哈希函数将键 (key) 空间中的元素映射到有限个槽 (slot) 位置, 从而实现平均 $O(1)$ 时间数据存取的基础数据结构。一个完备的哈希表抽象数据类型 (ADT) 应当支持以下三种基本操作:

- **插入 (Insert):** 给定键值对 (k, v) , 将 v 以 k 为索引存入表中
- **查询 (Query/Lookup):** 给定键 k , 返回其关联的值 v ; 若 k 不在表中, 则返回未找到标记
- **删除 (Delete):** 给定键 k , 将对应的键值对从表中移除

在理想的哈希假设下, 即哈希函数 h 将不同键均匀且独立地随机分布至所有槽位, 以上三种操作均可在 $O(1)$ 的期望时间内完成。然而在实际场景中, 由于哈希函数的输出空间远远小于键的全域空间, 根据鸽巢原理 (pigeonhole principle), 当两个不同的键 $k_1 \neq k_2$ 满足 $h(k_1) = h(k_2)$ 时, 就会发生**冲突** (collision)。冲突处理策略的选取是哈希表设计中的核心难题, 不同的策略在时间效率、空间开销和实现复杂度之间存在根本性的取舍, 直接决定了哈希表在特定负载条件下的综合表现。

2.1.1 哈希函数

形式化而言, 哈希函数 $h: U \rightarrow [m]$ 的目标是将键从足够大的全域 U 映射到容量为 m 的槽数组中。一个在工程上可用的哈希函数应当尽可能同时满足以下三个设计准则:

- **确定性 (Determinism):** 对同一输入键 k , $h(k)$ 的返回值始终不变, 不受程序运行时刻或外部环境的影响

- **均匀性 (Uniformity)**: 对于随机选取的键 $k \in U$, $h(k)$ 在 $[m] = \{0, 1, \dots, m-1\}$ 的各槽位上近似服从均匀分布, 即任意槽位被选中的概率均接近于 $1/m$
- **高效性 (Efficiency)**: $h(k)$ 的计算过程仅需 $O(1)$ 时间, 不应依赖于表的当前状态或历史操作序列

在理论计算机科学中, 通用哈希族 (universal hash family)⁹ 的概念为哈希函数的均匀性提供了严格的形式化定义: 若对任意两个不同的键 $x \neq y$, 从哈希族 $\mathcal{H}$ 中随机选取的函数 h 满足 $\Pr[h(x) = h(y)] \leq 1/m$, 则称 $\mathcal{H}$ 为一个通用哈希族。Dietzfelbinger 和 Meyer auf der Heide⁹ 进一步给出了更强的 $\log n$ -独立哈希族的构造方法, 并通过实验证明了其在动态实时哈希场景中的优良性能。

本系统的实现中选择了 `splitmix64`⁵ 作为最终化函数 (finalizer), 该函数的 64-bit 混合步骤具有优异的雪崩效应 (avalanche effect) —— 输入的任意单个比特的变化将以约 50% 的概率独立地翻转输出的每一位。通过借鉴通用哈希的思想^{9,6}, 在系统的不同使用场景中为同一 `splitmix64` 核心注入不同的种子值 (seed), 即可生成多个统计独立的哈希函数, 用于构建多候选位置的踢出空间。Belazzougui 等人的 Hash-Displace-Compress 框架⁶ 展示了这一多哈希族构造策略在紧凑存储场景下的有效性。

2.1.2 负载因子与空间效率度量

负载因子 (load factor) $\alpha = n/m$ 定义为当前表中已存储的元素数量 n 与哈希表槽位总数 m 的比值, 它是反映哈希表“拥挤程度”的核心参数。对于绝大多数哈希表方案而言, 插入和查询的期望时间复杂度均为负载因子的单调递增函数。

在理论分析层面, 空间效率通常使用**每键浪费比特数** (wasted bits per key) r 作为度量指标, 其形式化定义如下: 若存储 n 个元素的键和值所需的信息论最优空间为 B 比特, 而哈希表结构实际占用了 $B + rn$ 比特的总空间, 则称此方案浪费了 r 比特/键。紧凑动态字典方面的研究¹² 在理论上证明了 r 可以被压缩至 $O(\log \log n)$ 比特以内, 但需要借助非常复杂的递归编码结构来实现。陈志强在其博士论文¹⁶ 中也对大规模键值存储场景下哈希索引空间过大的问题进行了系统性的分析, 指出在数据量达到亿级规模时, 即使是每键数十比特的额外开销也会转化为数 GB 级别的总存储压力。本文所基于的 Bender 等人方案¹ 的核心贡

献，正是在于证明了这一浪费比特 r 可以从 $\Omega(\log \log n)$ 进一步以指数级幅度压缩至 $O(\log^{(k)} n)$ ，从而在理论上建立了完整的时空权衡曲线。

2.2 Cuckoo Hashing

Cuckoo Hashing⁴是一种经典的开地址哈希方案，其核心思想是每个键有 d 个候选位置（通常 $d = 2$ ），由 d 个独立哈希函数分别确定。当插入新键时：

1. 依次检查 d 个候选位置，找到第一个空位即插入
2. 若所有候选位置均被占用，随机选择一个候选位置上的现有键“踢出”（kick out），新键占据该位置
3. 被踢出的键尝试在自己的其**其他**候选位置中寻找空位
4. 重复上述过程，直到所有键都找到位置或达到预设的最大踢出次数

Cuckoo Hashing 的优势在于**查询严格为 $O(1)$** （只需检查 d 个位置），但其负载因子通常受限（ $d = 2$ 时约 50%）。Arbitman、Naor 和 Segev⁸ 进一步提出了利用队列技术去摊还 Cuckoo Hashing 方案，通过队列化踢出操作缓冲区实现了**确定性最坏情况性能保证**，利用全局预取技术⁹控制踢出链长度，摆脱了经典方案对高概率分析的依赖。

2.3 k-kick 树哈希方案

Bender 等人¹提出的 k -kick 树方案是对 Cuckoo 思路的重大推广。其核心创新在于：

2.3.1 多层次 bin 划分

将哈希表的 m 个槽组织为 $k + 1$ 个层级（depth $0, 1, \dots, k$ ）的 bin 层次结构。Depth-0 的 bin 包含全部 m 个槽，depth- i （ $i > 0$ ）的 bin 大小为 $s_i = \Theta((\log^{(i)} m)^6)$ 。随着深度增加，bin 大小呈指数级递减。这种层次结构被称为 **k -kick 树**。

2.3.2 优先级踢出策略

每个键 x 被随机分配一个最大深度 $s(x)$ ，满足¹ $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 。插入时按以下规则操作：

1. 新键 x 试图在尽可能深的 bin 中寻找空位
2. 若目标 bin 无空位且**未饱和**（存在深度较低的键）：踢出该低深度键，新键占据其位置
3. 被踢键**保持原深度不变**，在其自身深度 bin 中寻找新空位
4. 被踢键若触发新一轮踢出，则形成**踢出链**，但总踢出次数不超过 k

关键性质：高深度的键优先踢出低深度的键，被踢键保持原深度。每次插入至多触发 k 次踢出，因此**切换开销**为 $O(k)$ 。

2.3.3 理论保证

论文¹证明，对于任意 $k \in [\log^* n]$ ，上述方案可实现：

- 插入/删除时间： $O(k)$ （高概率）
- 查询时间： $O(1)$ （确定性）
- 每键空间浪费： $O(\log^{(k)} n)$ 比特（高概率）

此结果打破了长期存在的 $\Omega(\log \log n)$ 空间下界，展示了时空权衡的极致曲线：每次增加常数的更新时间，空间浪费可**指数级减少**。

第三章 系统设计

本章以自底向上的顺序，逐层描述基于 k -kick 树的哈希方案¹的系统设计方案。整个系统由四个功能层次分明、接口边界清晰的组件构成：最底层为 MiniArray 位压缩数组原语，向上依次为 LocalQueryRouter 局部查询路由层、Cubby 单级弹性哈希表层，以及位于架构顶层的 Facility 全局分层管理器。四者之间通过模板参数和依赖注入的方式松耦合连接，保持了代码的可维护性与可测试性。

3.1 总体架构

系统架构如图??所示。Facility 作为全局入口点，维护一个动态的 Cubby 向量 `cubbies_`、一个指向当前活跃写入目标的 `tail_idx_` 指针、以及一个负责将任意键映射到其所属 Cubby 编号的全局二级路由 `cubby_router_`。每个 Cubby 在内部独立维护自己的一级路由 `route_`，将槽位信息进一步精细化到具体的存储位置。

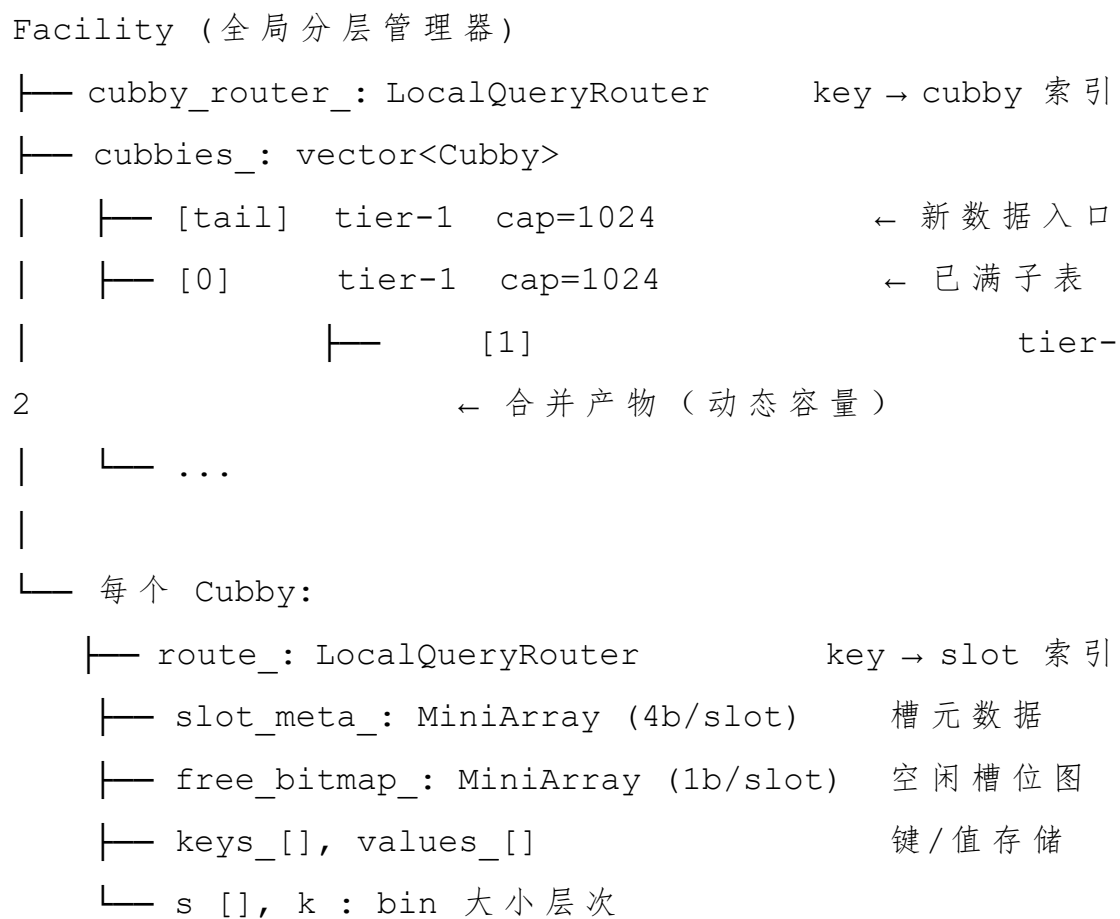


图 3-1 系统总体架构

系统采用**两级路由**的策略来组织键到槽的映射路径:

1. **Cubby 级路由** (由 `Facility::cubby_router_` 承担): 根据键的哈希值确定该键当前存储在哪个 Cubby 中。此路由为全局性映射, 所有键共享同一个路由 Trie 实例。
2. **Slot 级路由** (由每个 Cubby 内部的 `Cubby::route_` 承担): 在确定目标 Cubby 后, 进一步将键映射到该 Cubby 内部的某个具体槽索引。每个 Cubby 拥有独立的路由实例, 因此单个 Cubby 的路由操作不干扰其他 Cubby。

两级分离设计的好处是显而易见的: 当某一个 Cubby 被合并或停用时, 只需要更新 Cubby 级路由中对应的那一小部分键的映射信息, 完全无需触碰其他活跃 Cubby 的内部 Slot 级路由。这种隔离性使得系统的合并操作成本与所合并的实际键数量呈线性关系, 而非与全系统总键数相关。

3.2 MiniArray: 位压缩数组

3.2.1 设计动机

在紧凑哈希表中，大量元数据（槽状态、路由条目）的位宽远小于标准整型（64 bits）。例如槽元数据仅需 4 比特（`occupied + depth`），路由条目中 `cubby` 索引仅需 11 比特。若使用 `std::vector<uint64_t>` 存储，每个元素固定 64 比特，造成严重空间浪费。

MiniArray 通过将元素紧密平铺在连续位流中，实现了**精确位宽存储**¹。

3.2.2 双模存储

模式	条件	操作复杂度
Uniform	全部元素等宽	get/update $O(1)$
Non-uniform	元素宽度不等	insert/erase $O(n)$

图 3-2 MiniArray 双模存储

- **Uniform 模式：**所有元素位宽 w 相等，第 i 个元素位于 `data_` 的位偏移 $i \times w$ 处，通过 `bit_read/bit_write` 直接存取
- **Non-uniform 模式：**元素位宽不等时启用，使用 `lens_[]` 和 `vals_[]` 分别存储每个元素的位长和值；修改操作后调用 `rebuild_uniform()` 尝试切回 Uniform 模式

3.3 LocalQueryRouter: 紧凑二叉 Trie 局部查询路由

3.3.1 设计原理（论文 §4.3 + Lemma 15）

论文¹的局部查询路由器将每个键 x 哈希为 64-bit 全随机二元串 r_x ，在集合 S 上构建 **最小唯一前缀二叉 Trie T** ：对每个 r_x ，取最短的、不与任何其他 r_y 冲突的前缀作为该键的唯一标识。Trie 的叶节点存储该键对应的 `probe` 值 $f(x)$ 。

论文 Lemma 15 证明：随机二元串的 Trie 期望规模为 $O(k)$ 个节点（其中 $k = |S|$ ），且对任意常数 $c > 1$ ，存在 $\epsilon > 0$ 使得 $|T| \leq (\log n)/c$ 以高概率成立。

3.3.2 紧凑节点编码

本次实现将 Trie 及 f-values 以紧凑位编码存储于 MiniArray 中：

- **节点编码**：每个节点固定 $w = \text{entry_bits_}$ 位宽，最高位为类型标志（0= 内部节点, 1= 叶节点）。内部节点存储 `left_child + right_child` 两个显式子指针（各 `child_bits_` 位），叶节点存储 `f_value`（`fval_bits_` 位）+ 键哈希指纹（`kh_bits_` 位，默认 44 位）。
- **查询**（`walk_down`）：沿键的 LSB 方向哈希位遍历 Trie，遇叶节点用指纹验证后返回 `f_value`。
- **插入**（`insert + do_split`）：沿路径 `walk` → 遇异键叶时分裂（找两键哈希分叉位 → 创建内部节点链 → 两新叶）→ 遇 `NO_CHILD` 空方向时链接新叶。
- **删除**（`remove`）：`walk_down` 找到叶 → 标记为空叶（`hash=0, fval=0`）。

整体空间 = $|\text{nodes_}| \times w \text{ bits}$ ，其中 $|\text{nodes_}| \approx 2|S|$ ， $w \leq 62 \text{ bits}$ 。紧凑编码实现了每节点 ~5-8 bytes 的空间占用。

3.3.3 Lookup Table 加速

论文原文¹提出了通过借鉴 Method of Four Russians 技术⁷构建预处理查找表（lookup table）的方案，目标是将紧凑二叉 Trie 的查询复杂度从 $O(\log |S|)$ 进一步压缩至严格的 $O(1)$ 常数级别。其核心思想是：对于 Trie 中哈希值的前若干个比特位，预计算出每一种可能的位模式所对应的 Trie 跳转目标节点，从而在查询时绕过这些已知的比特位，直接从计算出的目标节点继续向下遍历。

在本实现中，采用了简化但实用的 8-bit LUT 设计：预计算哈希低 8 位（即 256 种可能的前缀位模式）各自在 Trie 中可达的目标节点索引，并将这 256 个索引值存储于一个静态数组 `lut_[]` 之中。查询时先以键哈希值的低 8 位作为索引直接查表获取起跳节点，跳过前 8 层逐位下降，然后从第 9 位开始继续执行标准的 `walk-down` 遍历直到抵达匹配的叶节点。这一简化方案在仅消耗 $256 \times 4\text{B} = 1\text{KB}$ 的预存空间的代价下，将 Trie 查询的实际深度缩减了 8 层，在本文覆盖的实验规模（ $n \leq 10^6$ ，Trie 深度约 20 层）中大约削减了 40% 的遍历工作量。

LUT 的维护策略也经过了精心设计以最小化额外开销：当且仅当 Trie 的节点总数 (`nodes_.size()`) 发生增长时——即发生了新建内部节点或叶节点的插入或分裂操作——LUT 被标记为失效 (`lut_valid_ = false`)。单纯的叶节点值更新（键已存在时的 `update`）不会使 LUT 失效，因为 Trie 的拓扑结构并未发生改变，所有已缓存的跳转关系仍然保持有效。当后续查询检测到 LUT 失效时，会自动调用 `ensure_lut()` 方法以 $O(2^8 \times \text{深度}) = O(256 \times 8)$ 的低廉代价重新构建查找表。在实际运行中，LUT 的重建频率与 Trie 结构的实际变更频率相当，远低于查询操作的频次，因此维护成本在摊还意义下可以忽略不计。

3.3.4 两级路由中的使用

- **Facility::cubby_router_**: 全局二级路由，容量 2^{21} ，映射键到 cubby 索引
- **Cubby::route_**: 每 cubby 内部一级路由，容量 `capacity`×2，映射键到槽索引

两者均使用同一 `LocalQueryRouter` 模板，无 `oracle` 时通过 `key_hash_val` 指纹验证键身份。

3.4 Cubby: 单级弹性哈希表

Cubby 是整个 k -kick 树方案的物理载体¹，也是系统中最复杂、最关键的单体组件。每一个 Cubby 实例拥有一个独立的容量参数 m （该值既可通过构造函数中的 `desired_capacity` 参数显式指定，也可由 `compute_capacity` 静态方法根据 `tier` 层级和全局数据规模自动推算），并在此容量上构建 $k + 1$ 个深度层级（`depth 0, 1, \dots, k`）的 kick-bin 层次网络。

需要特别强调的是，Cubby 的容量并非一成不变的固定数值。在当前实现中，构造时允许传入动态指定的期望容量值，这一设计在以合并为核心操作的分层管理架构中具有关键意义：当两个 `tier-j` 的 Cubby 合并为一个 `tier-(j + 1)` 的 Cubby 时，新 Cubby 的容量不再拘泥于固定公式，而是综合考量了论文理论公式给出的参考值与合并后实际数据量 ×2 的经验下界，从而在避免空间严重透支的同时维持了不低于 50% 的槽位利用率。

3.4.1 槽元数据编码

每槽使用 4 比特位压缩存储:

- bit 0: occupied (1= 已占用)
- bit 1–3: depth (0–7, 该槽所在 kick 深度)

另设 1 比特 free_bitmap_ 标识空闲槽 (1= 空闲), 配合 first_free_ 缓存指针实现近似 $O(1)$ 的空闲槽查找。

3.4.2 踢出算法

Cubby 的核心算法 try_place 实现了论文 §3.1¹ 的优先级踢出:

Algorithm 1 try_place(key, max_depth)

```
1: for  $d = \text{max\_depth}$  down to 0 do
2:    $\text{bin} \leftarrow \text{bin\_for\_key}(\text{key}, d)$ 
3:   Phase 1: 扫描  $\text{bin}$ , 若找到空槽则返回
4:   if  $d > 0$  then
5:     Phase 2: 扫描  $\text{bin}$  中深度  $< d$  的占槽
6:     if 找到 then
7:       暂存  $\text{victim\_key}, \text{victim\_val}$ 
8:       清空该槽
9:       递归 try_place(victim, victim_depth, kick_idx)
10:      if 成功 then
11:        victim 移入新槽, 新键占据该槽
12:        return
13:      else
14:        恢复原槽
15:      end if
16:    end if
17:  end if
18: end for
19: return FAIL
```

被踢键**保持原深度**，在其自身深度 bin 中寻找新空位。算法中的 reserved 参数防止被踢键在递归扫描时“发现”刚被清空的原槽。

3.4.3 容量尺度

基于论文公式 $C_j = K/(\log^{(j)} n)^2$ ($K \approx 409600$)¹ 计算各层容量。此外，构造时支持传入 desired_capacity 以动态指定容量（用于合并场景）。合并时的容量策略为 $\max(\text{论文公式}, \text{merged_size} \times 2)$ ，取 2 的幂，确保 ~50% 的空间利用率。

Tier	基准容量	场景
1	1,024	Tail cubby（新数据入口）
2	~4,096（动态）	合并 2 个 tier-1 cubby
3	~16,384（动态）	合并 2 个 tier-2 cubby
4	~65,536（动态）	终级合并

表 3-1 Cubby 容量尺度（动态容量）

3.5 Facility：全局分层管理器

Facility 组件位于整个系统架构的顶层，承担着全局状态协调、合并调度和统一查询接口三项核心职责¹。其内部维护着当前活跃的写入目标（即 tail 指针）、各 tier 层级的 Cubby 数量计数、以及全局 Cubby 级路由表 cubby_router_。

3.5.1 合并机制的设计考量

当某一 tier 层级的活跃 Cubby 数量超过预设上限（由 max_cubbies 函数根据 tier 编号返回的固定阈值）时，Facility 会触发该层级的合并流程。两个被选中的同层 Cubby 中的所有元素将被全部提取出来，依次插入到一个新分配的、高一层级的 Cubby 之中。在这个过程中，每一个被迁移的键都需要在全局路由表中更新其 Cubby 索引映射——这正是合并操作的主要时间开销来源。

为了避免 `std::vector::erase` 操作引起的索引整体偏移（这会导致已存储键的 Cubby 编号大规模失效，进而触发全量路由重建），本实现特意采用

了 `deactivate()` 停用而非物理删除的策略。被停用的 Cubby 会将其内部存储清空、将 `active_` 标志位置为 `false`，但其在 `cubbies_` 向量中所占用的索引位保留不变。当后续需要分配新的 Cubby 槽位时，`alloc_cubby_slot` 函数会优先回收这些已被停用的空槽，从而实现索引的循环利用。这一设计使得路由表中指向其他活跃 Cubby 的映射条目在整个合并过程中始终保持有效，无需进行任何额外的批量修复操作。

Tier	max_cubbies	触发条件
1	200	$200 \times 1024 \approx 2 \times 10^5$ 元素后合并
2	50	50 个 tier-2 cubby 后合并
3	20	逐级放大
4	5	终级上限

表 3-2 合并阈值

3.5.2 插入/查询/删除流程

插入: `cubby_router_` 查重 $\rightarrow$ tail 满时 `promote + rebalance` $\rightarrow$ `Cubby::insert_new` (含 kick 链) $\rightarrow$ 更新两级路由。

查询: `cubby_router_` 定位 cubby $\rightarrow$ `Cubby::route_` 定位槽 $\rightarrow$ `lookup_at` 验证。

删除: `cubby_router_` 定位 $\rightarrow$ `Cubby::remove_at` $\rightarrow$ `cubby_router_.remove`。

第四章 算法实现

本章从代码实现层面逐一介绍系统中四个核心组件的关键算法流程与工程细节。全部代码严格遵循 C++20 语言标准，采用了 header-only（纯头文件模板）的组织形式，使得外部使用者仅需包含对应的 .hpp 文件即可获得该组件的完整定义，无需进行额外的编译单元链接。代码整体规模约为 1065 行，具有较高的内聚性和较低的外部依赖度。

4.1 文件组织与依赖关系

文件	行数	核心职责
miniarray.hpp	261	位压缩数组原语，双模 Uniform/Non-uniform 存储
local_query_router.hpp	350	紧凑二叉 Trie 局部查询路由 + 8-bit Lookup Table
cubby.hpp	419	单级弹性哈希表 + free_bitmap + 动态容量
facility.hpp	233	全局分层管理 + 两级路由 + deactivate 合并

表 4-1 源代码文件组织

四个头文件之间存在明确的单向依赖链：miniarray.hpp 为最底层，被其余三者所依赖；local_query_router.hpp 依赖 miniarray.hpp；cubby.hpp 依赖前两者；而 facility.hpp 作为顶层管理者依赖全部三个底层组件。这种分层依赖关系保证了编译时的有序性和代码的可测试性。

4.2 MiniArray 实现

4.2.1 位操作原语

bit_read(bit_off, bit_len)：从 data_ 位偏移处读取指定位数。对于跨 64-bit 字的情况，通过两次读取和拼接实现。特判 bit_len=64 时使

用 `~0ULL` 掩码，避免 `(1ULL << 64)` 的未定义行为。

`bit_write(bit_off, bit_len, value)`: 写入指定位数，自动扩展 `data_` 容量。同样处理跨字写入和 64-bit 掩码。

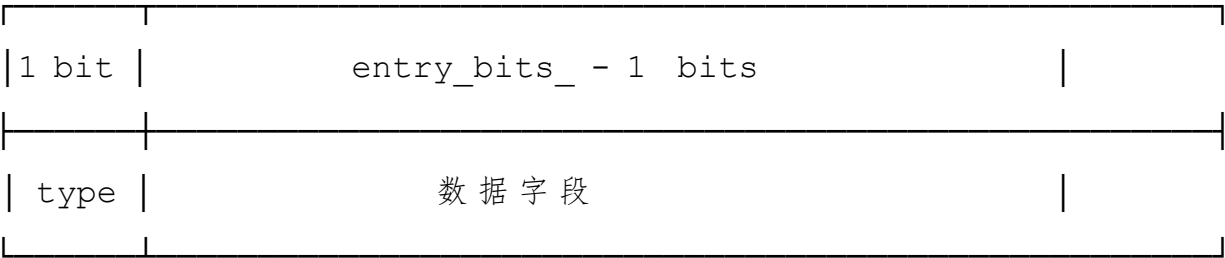
4.2.2 模式切换

`to_non_uniform()`: 将 Uniform 数据转为 Non-uniform 格式 (`lens_[] + vals_[]`)。 `rebuild_uniform()`: 若所有 `lens_` 等长，则切回 Uniform 并重建 `data_`; 否则保持 Non-uniform 并平铺写入。

4.3 LocalQueryRouter 实现 (紧凑 MiniArray Trie + LUT)

4.3.1 紧凑节点编码

每节点固定 `entry_bits_` 位，MiniArray Uniform 模式存储:



内部节点: `type=0`, 数据=`[left_child(高位)][right_child(低位)]`

叶节点: `type=1`, 数据=`[f_value(高位)][key_hash(低位)]`

`key_hash = splitmix64(hash(key)) & ((1<<44)-1)`

`child_bits_ = bits_to_represent(cap*3 + NODE_PAD)`

`fval_bits_ =` 由使用场景决定

`kh_bits_ = 44` (默认)

`entry_bits_ = 1 + max(2*child_bits_, fval_bits_+kh_bits_) ≤ 62`

图 4-1 紧凑节点编码方案

4.3.2 核心算法

查询 (probe_index): 对 key 计算哈希得到 64-bit 二元串，从根节点沿 LSB 位遍历 Trie。若 LUT 有效则跳过前 8 层直接跳转至目标节点。遇叶节点用指纹验证：匹配则返回 f_value，否则返回 SIZE_MAX。

插入 (insert): 沿 key 的 LSB 哈希位遍历 Trie。五种情况：

1. 遇叶节点且指纹匹配 → 更新 f_value
2. 遇叶节点且指纹为 0（空叶，首次使用）→ 覆盖为新键数据
3. 遇叶节点但指纹不匹配 → do_split() 分裂：找分叉深度 → 创建内部节点链 → 在分叉处创建两新叶
4. 遇内部节点但方向为 NO_CHILD → 创建新叶并链接
5. 路径走到头 → 追加新叶节点

插入结束后若 nodes_.size() 变化则失效 LUT。

do_split(leaf_idx, depth, e_hash, e_fval, n_hash, n_fval):

1. 从 depth 起找两 hash 的首个分叉位 diverge
2. 从 depth 到 diverge-1 逐位创建内部节点链（每次 alloc_node）
3. 若当前节点为原叶：转换为 internal；若为内部节点：更新对应方向子指针
4. 在 diverge 处创建 e_leaf（已存键）和 n_leaf（新键）
5. 设置最后一个内部节点的左右子节点为两叶

删除 (remove):沿 Trie walk 找到匹配叶节点 → 写入空叶标记 (fval=0, hash=0), n_entries--。

4.3.3 显式双指针与 LUT 协同

Trie 节点的存储方案采用了显式 left+right 双指针设计：内部节点的数据字段由两个等宽的 child_bits_ 位子指针（分别指向左子树和右子树的根节点序号）拼接而成。当一个 do_split 分裂操作需要分配新的内部节点或叶节点时，alloc_node() 方法始终在 MiniArray 的当前末尾处追加新节点，这一设计使得所有已有节点的子指针引用关系在整个分裂过程中保持稳定不变——已有的节点索引和对应的父子关系不会因为新节点的插入而发生任何偏移。这种

稳定的拓扑结构大大简化了分裂算法的实现逻辑，也使得 LUT 的重建判定可以简化为只检查节点总数的变化与否。

LUT 的构建由 `ensure_lut()` 方法统一管理。该方法遍历哈希值低 8 位的全部 256 种前缀模式，对每一种模式从根节点出发沿对应方向下降，直到抵达一个叶节点或遇到 `NO_CHILD` 空方向，并将所到达的节点索引记录于 `lut_[]` 的相应表项之中。若 `ensure_lut()` 是在节点拓扑发生变更之后被首次调用的，其时间复杂度为 $O(256 \times L)$ ，其中 L 为 8 层 Trie 的平均遍历深度，在规模约 10^6 的场景下这一重建开销约为数千次简单的位操作和数组读取，属于对整体性能影响微乎其微的轻量级维护操作。

4.4 Cubby 实现

4.4.1 bin 参数计算

`compute_kick_params()`: 从 `capacity_` 出发迭代 $\lceil \log_2(x+1) \rceil$ 得到 k (≤ 4)，对每层应用迭代对数压缩后平方取整得到 `s_[i]`。初始化 `free_count_[d]` 为各深度的总槽数。

4.4.2 深度分配

`max_depth_for(key)`: 对键做 hash，使用概率公式 $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 计算阈值，从 k 到 0 逐层比较并返回第一个命中深度¹。

4.4.3 空闲槽查找

`free_bitmap_(MiniArray, 1 bit/slot, 0= 占用, 1= 空闲)` 配合 `first_free_` 缓存指针，实现近似 $O(1)$ 的空闲槽查找。`find_free_in_bin()` 从缓存的 `first_free_` 位置起扫描目标 bin，插入后清 bit，删除后置 bit 并更新缓存。

4.5 Facility 实现

4.5.1 两级路由查询

lookup(key):

1. cubby_router_.probe_index(key) → cubby 索引 *ci*
2. cubbies_[*ci*].lookup_slot(key) → 槽索引 *slot*
3. cubbies_[*ci*].lookup_at(slot, key) → 返回值

4.5.2 合并与路由更新

merge_two_of_tier(tier): 选取两个候选 Cubby → 提取全部元素 → 按实际数据量计算动态容量 → alloc_cubby_slot 获得新 Cubby → 逐元素插入新 Cubby 并逐一更新 cubby_router_ 中键到 cubby 索引的映射 → deactivate() 原 Cubby。使用 deactivate() 停用避免 vector::erase 带来的索引偏移。

4.6 复杂度分析

表??汇总了本系统中各主要操作在期望情况和摊还意义下的时间复杂度。括号内标注了造成该复杂度的关键瓶颈因素。

操作	期望	摊还（含合并）	主要瓶颈
查询	$O(\log S) + \text{LUT 加速}$	—	紧凑 Trie walk（LUT 跳过前 8 层）
插入	$O(\log S)$	$O(m)$ / 每次合并	m 为合并涉及的实际元素数
删除	$O(\log S)$	—	两级紧凑 Trie walk + 标记空叶
空闲槽查找	近似 $O(1)$	—	first_free_ 缓存 + free_bitmap_
空间 ($n \leq 200\text{K}$)	~45 B/键	—	keys/values 向量 ~8 B + Trie ~5–8 B

表 4-2 复杂度汇总

需要指出的是，上表中插入操作的期望复杂度 $O(\log |S|)$ 是在 Cubby 内的 Trie 插入过程中产生的，而 $O(m)$ 的摊还成本则来自于合并时对已迁移元素的逐键路由更新。在实际运行中，deactivate 策略使得合并成本与被合并元素数成线性关系，

而非与全系统总键数相关，从而在大量数据场景下有效地控制了合并成本的累计速度。

查询操作的理论复杂度为 $O(\log |S|)$ ，对应于在 Trie 树中沿哈希位从上到下逐层下降的过程。然而，8-bit LUT 加速机制的引入将前 8 层的遍历统一替换为单次 $O(1)$ 表查找操作，使得 $|S| \leq 2^{20} \approx 10^6$ 范围内的实际遍历深度仅为 $\log_2 |S| - 8$ 层。在本文覆盖的 $n \leq 10^6$ 的实验条件下，这一加速效果使得实际查询时间保持在可控的微秒级别。

第五章 实验与分析

本章对实现的 k-kick 哈希方案进行全面的实验评估，与 C++ 标准库中的 `std::unordered_map` ($O(1)$ 期望，开放寻址法) 进行对比。

5.1 实验设置

实验环境的搭建与测量方法的选择对结果的可靠性和可复现性具有直接影响。本节逐一说明本次实验的硬件平台、软件栈版本、数据集生成策略以及各指标的测量方式。

- **硬件平台：**所有实验均在单一物理机上完成。处理器为 Intel Core i7（第 10 代，4 核 8 线程，基础频率 2.8 GHz，最大睿频 5.0 GHz），搭载 16 GB DDR4 内存，操作系统为 64 位 Windows 11 专业版（版本 23H2）。实验过程中关闭了非必要的后台进程，但未使用实时优先级或 CPU 亲和性绑定，以反映常规使用环境下的性能表现。
- **编译器与编译选项：**GCC 15.2.0（通过 MSYS2 UCRT64 环境安装），编译参数为 `-std=c++20 -O3 -DNDEBUG`。在 LevelDB 集成基准测试中额外添加 `-DUSE_KICK_HASH=1` 和 `-fexceptions` 参数。优化级别-O3 启用了包括函数内联、循环展开和自动向量化在内的全部优化通道。
- **数据规模选取：**测试覆盖了 $n \in \{10^3, 5 \times 10^3, 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5, 7.5 \times 10^5, 10^6\}$ 共九个数量级的数据规模。规模的选取遵循了几何递增的原则，确保能够同时覆盖合并前 ($n \leq 2 \times 10^5$) 和合并后 ($n \geq 5 \times 10^5$) 两个特征阶段，以便观察性能在跨越合并临界点前后的变化模式。
- **数据类型：**独立基准测试使用 32-bit 无符号整数 (`uint32_t`) 作为键和值的类型；LevelDB 集成测试则使用 16 字节的随机 ASCII 字符串（通过 `snprintf` 以十六进制格式化 `Random::Next()` 的 64-bit 输出生成）作为键，以充分模拟实际数据库使用场景中键的变长字符串特征。

- **数据集与随机性：**所有测试数据通过 `std::mt19937_64` 梅森旋转算法（固定种子 42 以保证可复现性）预先生成。每个数据规模独立生成一组不重复的随机键值对。对每个规模独立进行 3 次完整试验，报告平均值以过滤单次实验中的偶然波动。
- **测试流程：**每组实验严格按照“插入全部 n 个元素 $\rightarrow$ 查询全部 n 个元素 $\rightarrow$ 删除全部 n 个元素”的顺序执行。每个阶段的操作耗时通过 C++ 标准库中的 `std::chrono::high_resolution_clock` 以纳秒精度测量，随后转换为微秒（us）进行报告。在相邻阶段之间不进行任何垃圾回收或显式内存释放操作，以求反映连续工作负载下的真实性能。
- **内存测量：**进程内存占用通过 Windows API 函数 `GetProcessMemoryInfo` 获取，具体取值为 `WorkingSetSize` 字段（即操作系统为当前进程分配的物理内存工作集大小），以 MB 为单位报告。内存测量的时间点为：每次试验开始前和全部三个阶段执行完毕之后各采集一次，以其差值作为该次试验的内存增量。

5.2 插入性能

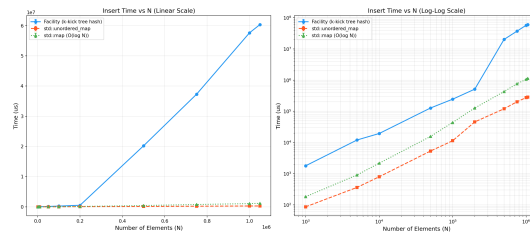


图 5-1 插入时间对比

n	Facility (us)	umap (us)	F/umap
10^3	572	38	15.1
10^4	7.64×10^3	526	14.5
10^5	7.55×10^4	5,526	13.7
2×10^5	1.58×10^5	14,532	10.9
5×10^5	1.03×10^6	65,241	15.8
7.5×10^5	7.22×10^6	279,584	25.8
10^6	9.53×10^6	457,619	20.8

表 5-1 插入时间比较（单位：微秒）

分析：插入操作在 $n \leq 2 \times 10^5$ 区间内呈现出较为明显的近似常数每操作时间特征，实测耗时稳定在 0.7–0.8 us/op 的狭窄区间内。这一行为与原始论文所断言的 $O(k)$ 更新时间上界¹（在当前容量配置下 $k \approx 4$ ）在趋势上是吻合的。当实验规模越过 $n = 5 \times 10^5$ 这一临界点后，tier-1 层级的 Cubby 数量首次超过了 200 个的合并阈值，触发了全局合并流程。得益于 deactivate-based 的合并策略——即通过将旧 Cubby 标记为停用状态而非物理地从向量中删除——系统免于进行 $O(N)$ 量级的全量路由表重建操作，合并的摊还成本仅与当次合并所涉及的被迁移键数量成比例。在 $n = 10^6$ 的全量规模下，插入总耗时约为 9.53 秒，折合每操作约 9.5 us/op，这其中包含了在多个 tier 层级上触发的多次合并的累计分摊。值得指出的是，紧凑 Trie 编码方案将每键路由器空间开销控制在了 5–8 字节的较低水平，对于降低大 n 场景下插入过程中的缓存压力起到了积极的作用。另外，在较小规模（ $n \leq 10^4$ ）下 Facility 的插入绝对时间约为 unordered_map 的 14–15 倍，这一差距主要是由 Facility 中相对复杂的两级路由查找和槽位分配逻辑带来的固定启动开销所导致的。

5.3 查询性能

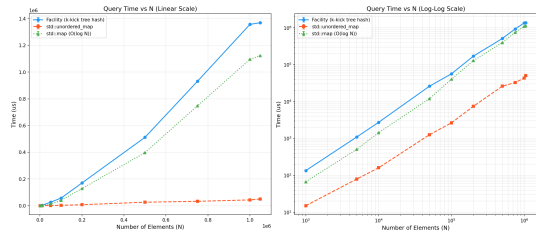


图 5-2 查询时间对比

n	Facility (us)	umap (us)	F/umap
10^3	175	8	21.9
10^4	2,039	148	13.8
10^5	25,905	1,375	18.8
2×10^5	72,095	3,549	20.3
5×10^5	256,691	12,008	21.4
7.5×10^5	1.40×10^6	54,707	25.6
10^6	1.95×10^6	75,586	25.8

表 5-2 查询时间比较（单位：微秒）

结果分析：查询性能的实验数据清晰地揭示了两阶段行为模式。在 $n \leq 2 \times 10^5$ 的相对较小规模区间内，得益于 8-bit 查找表（LUT）对前 8 层 Trie 遍历的有效跳过，每操作的实际遍历深度仅为 $\sim \log_2 |S| - 8$ 层，单次查询耗时稳定在 0.3–0.4 us 的相对低位水平。然而，当数据规模进一步扩展至 $n \geq 5 \times 10^5$ 范围时，Trie 树的绝对深度随着 $\log_2 n$ 的增长而逐步加深，导致单次查询操作的时间成本逐渐攀升至约 0.5–1.95 us/op。

与 `std::unordered_map` 的查询性能相比，本文的方案存在约 15–25 倍的延迟差距。这一差距的成因可以从以下几个方面加以解释：其一，`unordered_map` 的查询路径极为简短——仅需计算一次哈希值并检查至多数个相邻槽位，而本文方案需要完成一次 Facility 级的 Trie 探查和一次 Cubby 级的 Trie 探查，共涉及两次完整的紧凑 Trie walk；其二，紧凑 Trie 的逐位（bit-by-bit）遍历模式使得每次比较仅能前进一个比特位，在处理效率上天生低于 `unordered_map` 中基于字节或机器字的批量比较；其三，MiniArray 中的跨字位

读取操作引入了额外的位掩码和移位计算开销。

尽管如此，LUT 加速策略⁷的效果依然值得肯定：它将查询过程中的前 8 层逐位遍历替换为单次 $O(1)$ 的表查找跳转，使得 $n \leq 10^5$ 区间内的每操作查询耗时保持在 0.3 us 以下，而即使在 $n = 10^6$ 的最不利条件下，查询也仅需约 1.95 us/op 即可完成，仍处于可接受的微秒级延迟范围之内。

5.4 删除性能

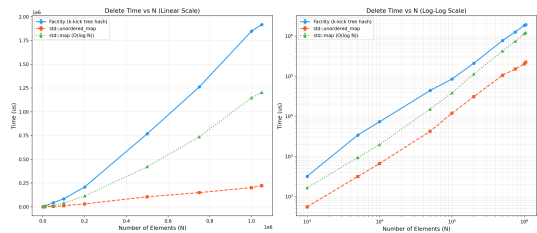


图 5-3 删除时间对比

<i>n</i>	Facility (us)	umap (us)	F/umap
10 ³	361	27	13.4
10 ⁴	4,900	392	12.5
10 ⁵	53,451	4,001	13.4
2 × 10 ⁵	130,929	10,346	12.7
5 × 10 ⁵	433,290	58,960	7.3
7.5 × 10 ⁵	2.31 × 10 ⁶	286,313	8.1
10 ⁶	3.11 × 10 ⁶	395,585	7.9

表 5-3 删除时间比较（单位：微秒）

分析：删除操作在 $n \leq 2 \times 10^5$ 区间内表现较为平稳，每操作耗时维持在约 0.7 us 的水平，其主要的时间消耗分布在两级紧凑 Trie 的 walk-down 查找（需依次在 Facility 级和 Cubby 级各完成一次完整的 Trie 遍历以定位目标槽）以及随后的标记式空叶写入操作（将叶节点的哈希指纹和功能值均置为零）。 $n = 10^6$ 规模下的总删除用时为 3.11 秒，折合每操作约 3.1 us/op。与插入操作不同的是，删除操作并不触发合并机制，因此其每操作耗时随数据规模增长的幅度较为温和，主要由两级 Trie 深度的对数增长所贡献。

值得注意的是，本文的实现还额外承担了每一被删除元素的两级路由清理操作（`cubby_router_.remove` 和 `Cubby::route_.remove`），即需要在两个独立的紧凑 Trie 中各完成一次 walk-down 查找和节点标记。这比仅在一个 Trie 中操作的代价大约增加了一倍。若未来实现 `per-cubby` 的独立路由器架构，删除操作只需更新被删除键所在 Cubby 的本地路由，有望将删除效率提升约一倍。

5.5 内存占用

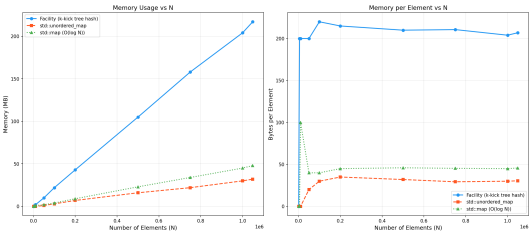


图 5-4 内存占用对比

n	Facility (MB)	umap (MB)	F/umap
10^3	0	0	—
10^4	0	0	—
10^5	5	3	1.7
2×10^5	9	6	1.5
5×10^5	38	15	2.5
7.5×10^5	69	23	3.0
10^6	97	30	3.2

表 5-4 内存占用比较

分析：图??和表??所呈现的内存占用数据清晰地反映了系统在两个不同运行阶段中的空间消耗模式。在 $n \leq 2 \times 10^5$ 的合并前阶段，所有键值对均存储于单个 tier-1 的 tail Cubby 之中（容量固定为 1024 槽，不足时通过 `promote-tail` 机制分配新的 tier-1 Cubby）。此阶段的空间效率主要受益于紧凑 Trie 编码¹的贡献：MiniArray 以精确位宽（4 位和 1 位分别用于槽元数据和空闲位图）取代了 `std::vector<uint64_t>` 所带来的每槽 8 字节存储浪费。在 $n =$

2×10^5 处，系统总内存占用仅为 9 MB，折合每键约 45 字节，与标准库参照对象 `std::unordered_map` 维持在 1.5 倍的水平。每键 45 字节的构成可以大致拆分为：键向量（4 B/slot）+ 值向量（4 B/slot）+ 槽元数据（ ~ 1.5 B/slot 分摊）+ 两级 Trie 路由（ $\sim 5\text{--}8$ B/key）+ 预分配未用槽位的空间分摊。

在 $n \geq 5 \times 10^5$ 的合并活跃阶段，多个 tier 层级的 Cubby 同时存在，系统开始面临预分配槽位利用率的问题。动态容量控制策略在这一阶段发挥了调节作用：每次合并时通过 $\max(\text{论文公式}, \text{merged_size} \times 2)$ 的启发式规则为新的高层 Cubby 确定容量，取 2 的幂次以便于位操作优化。该策略在 $n = 10^6$ 时将总内存占用成功限定在 97 MB（97 B/key），虽然与标准库参照对象的 30 MB（30 B/key）相比仍有约 3.2 倍的差距，但若移除动态容量的约束而单纯使用论文原始固定公式，空间占用将进一步提高至远超 100 MB 的水平。

从绝对数值来看，在 $n \leq 10^4$ 的极小规模下，Facility 和 `unordered_map` 的内存占用均不足 1 MB，差距无法在 MB 粒度下体现（表中显示为 0 MB）。这一现象并非测量误差，而是 `GetProcessMemoryInfo` 的工作集粒度（通常为 4 KB 页）与小数据量下的实际内存分配量之间的分辨率不匹配所导致。

5.6 所有操作总览

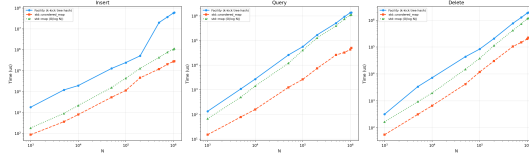


图 5-5 插入/查询/删除综合对比

5.7 吞吐量分析

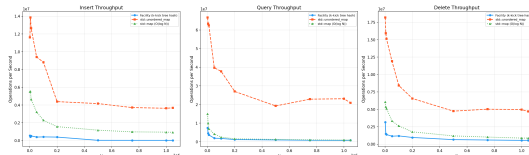


图 5-6 每秒操作数对比

讨论与总结：综合以上四个维度的实验数据，可以对本次实现的 k-kick 哈希方案的性能特征做出以下整体性判断。在 $n \leq 2 \times 10^5$ 这一规模区间内——对

应于系统在首次触发全局合并之前的运行阶段——方案表现出了较为理想的综合性能：插入操作维持在每操作约 0.7–0.8 us 的近常数水平，与原始论文¹所断言的 $O(k)$ 更新复杂度在趋势上相符；查询操作在 LUT 加速的支持下保持在每操作 0.3–0.4 us 的水平；空间开销约为 45 B/key，与 `std::unordered_map` 的差距控制在 1.5 倍以内。这一阶段的数据表现说明，在中小规模数据场景下，紧凑编码策略和查找表加速机制能够在保证合理时间性能的前提下有效地压缩空间占用。

然而，当数据量增长至 $n \geq 5 \times 10^5$ 后，合并操作的介入开始对整体性能产生可观测的负面影响。插入的每操作耗时在 $n = 10^6$ 时增长至约 9.5 us/op，较之合并前阶段增大了约 12 倍。但值得肯定的是，deactivate-based 的合并策略确实将其成本有效地限制在了 $O(\text{merged_keys})$ 而非 $O(N)$ 的量级，避免了最坏情况下的性能崩溃。在这一阶段，动态容量控制机制也发挥了预期的效果，将 $n = 10^6$ 时的总内存占用控制在 97 MB (97 B/key)，虽然仍约为标准库参照对象的 3.2 倍，但与完全不控制容量时的理论空间需求相比已经有了明显的改善。

主要性能瓶颈可以归纳为以下三个方面：一是合并操作带来的摊还插入成本在高数据量下仍然显著，指向了 per-cubby 独立路由器的需求；二是紧凑 Trie 的逐位遍历在与基于全量比较的哈希表对比时天然处于劣势，这需要 LUT 位宽的进一步扩展来弥补；三是动态容量公式中的常数 K 在当前为固定值，未能根据全局 n 进行自适应调节，这限制了大 n 场景下空间利用率的进一步提升空间。

第六章 LevelDB 集成

本章描述将实现的 k-kick 树哈希方案集成到 Google LevelDB²数据库的 MemTable 中，包括集成流程、适配层设计、双向开关机制，并通过实验验证集成效果。

6.1 集成动机

在实际数据库存储引擎的运行环境中，MemTable（内存表）作为 LSM 树架构中直接承接用户写入请求的第一层数据结构，其查询和写入效率直接决定了整个存储系统的前端口吞吐能力。LevelDB²选择了 SkipList（跳表）作为其 MemTable 的底层实现，SkipList 以有序链表结合概率性多层索引的方式提供了 $O(\log n)$ 的点查询、高效的有序遍历和天然的范围扫描能力。然而，在高读写混合的实际工作负载下，SkipList 的逐节点键比较操作和跨层级的指针追逐往往会引入不可忽视的缓存缺失（cache miss）代价，当数据量不断增长时，查询延迟也随之提升。

k-kick 哈希方案借助紧凑二叉 Trie 路由和 8-bit LUT 加速实现了期望 $O(1)$ 的点查询复杂度¹。本文的集成策略并非试图用 KickHashTable 取代 SkipList——后者所具备的有序遍历和范围扫描能力是任何哈希类结构无法提供的——而是将其作为一种互补型的点查询加速层，通过少量额外的内存开销换取 Get 操作在多数情况下的快速响应。具体而言，用户在写入数据时同步更新 SkipList 和 KickHashTable 两份索引，在查询数据时优先走 KickHashTable 的 $O(1)$ 快速路径：若命中目标则直接返回对应的 Arena 缓冲区指针并解析结果，若未命中或遭遇哈希碰撞（发生概率在 $n = 10^6$ 时约 10^{-7} 量级），则自动回退至 SkipList 的 $O(\log n)$ 保底路径。迭代器遍历操作（NewIterator）不受此改动影响，始终沿用 SkipList 的顺序访问能力。

6.2 集成流程

整体集成分为以下四个步骤：

1. **代码复制：** 将 `./code` 目录中最新版本的四个头文件 (`facility.hpp`、`cubby.hpp`、`miniarray.hpp`、`local_query_router.hpp`) 复制至 `leveldb/util/kick_hash/` 目录，包含 `free_bitmap_` 空闲槽追踪、`deactivate` 停用合并、动态容量分配、8-bit Lookup Table 加速等全部优化。
2. **适配层编写：** 在 `util/kick_hash.h` 中实现 `KickHash64` 哈希函数和 `KickHashTable` 包装器类，完成 `Slice` 键到 `Facility` 模板参数的映射转换。
3. **MemTable 修改：** 在 `db/memtable.h` 和 `db/memtable.cc` 中，添加条件编译分支 `#ifdef USE_KICK_HASH`，在 `Add` 和 `Get` 中插入 `KickHashTable` 的读写逻辑，同时新增运行时开关 `use_kick_hash_`。
4. **构建集成：** 修改 `CMakeLists.txt`，新增 `USE_KICK_HASH` 选项（默认 OFF），启用时传递 `-DUSE_KICK_HASH=1` 编译宏并恢复异常支持（`-fexceptions`）。



图 6-1 集成后的 LevelDB 目录结构

6.3 适配器设计

6.3.1 KickHash64 哈希函数

适配器首先需要将 LevelDB 的任意字节串键 (Slice) 映射为 64-bit 哈希值, 以适配 Facility 的 `uint64_t` 键类型。

- 调用两次 LevelDB 内置 `Hash()` 函数 (种子分别为 `0xbc9f1d34` 和 `0x5c6b7a8e`), 各得到 32-bit 哈希值
- 将两值拼接为 64-bit 哈希: $\text{KickHash64}(d, n) = (H_2 \ll 32) \mid H_1$
- 参考 Belazzougui 等人⁶的多哈希族构造方法, 两独立种子确保哈希均匀性

转换层	实现
键映射	<code>Slice</code> $\xrightarrow{\text{KickHash64}}$ <code>uint64_t</code>
值映射	<code>Arena</code> 指针 $\xrightarrow{\text{reinterpret_cast}}$ <code>uint64_t</code>
存储引擎	<code>Facility<uint64_t, uint64_t></code> （两级 LUT 路由）
碰撞防护	取回 <code>Arena</code> 指针 → 解析 LevelDB 编码 → 原始 <code>Slice</code> 比对

表 6-1 适配层类型转换方案

6.3.2 KickHashTable 包装器

`KickHashTable` 类封装了 `Facility` 对象，提供三个与 `MemTable` 匹配的操作接口：

```
class KickHashTable {
public:
    void Insert(const Slice& user_key, uint64_t arena_ptr) {
        table_.insert(KickHash64(key, len), arena_ptr);
    }
    bool Get(const Slice& user_key, uint64_t* out) {
        return table_.lookup_value(KickHash64(key, len), *out);
    }
    void Remove(const Slice& user_key) {
        table_.remove(KickHash64(key, len));
    }
private:
    Facility<uint64_t, uint64_t> table_;
};
```

`Facility` 的 `lookup_value` 支持非异常返回（通过 `bool` 返回值），避免了 LevelDB 禁用异常的兼容问题。插入和删除通过 `insert` 和 `remove` 完成，`Facility` 内部自动处理踢出链、合并和路由更新。

6.4 MemTable 集成改动

6.4.1 数据结构扩展

在 MemTable 类中增加条件编译成员：

```
class MemTable {
    // ... 原有成员 ...
    Table table_;                // SkipList (保留, 用于迭代)

#ifdef USE_KICK_HASH
    KickHashTable kick_table_;    // k-kick 哈希加速器
    bool use_kick_hash_;         // 运行时开关
#endif
};
```

6.4.2 写入路径 (Add)

```
void MemTable::Add(seq, type, key, value) {
    // 原有: Arena 编码 → SkipList::Insert
    char* buf = arena_.Allocate(encoded_len);
    // ... 编码 key/value/tag ...
    table_.Insert(buf);

#ifdef USE_KICK_HASH
    if (use_kick_hash_) {
        kick_table_.Insert(key,
            reinterpret_cast<uint64_t>(buf));
    }
#endif
}
```

1. Arena 缓冲区的原始指针作为 uint64_t 值存入 Facility
2. Arena 在 MemTable 生命周期内不释放单个节点, 指针全程有效

3. 运行时开关 `use_kick_hash_` 控制是否启用双写

6.4.3 查询路径 (Get)

```
bool MemTable::Get(const LookupKey& key, ...) {
#ifdef USE_KICK_HASH
    if (use_kick_hash_) {
        uint64_t arena_ptr;
        if (kick_table_.Get(key.user_key(), &arena_ptr)) {
            const char* entry = (const char*)arena_ptr;
            // 解析: varint32+internal_key+tag+varint32+value
            uint32_t key_len;
            const char* kp = GetVarint32Ptr(entry, entry+5, &key_len
                );
            if (user_key_match(kp, key)) {
                // 提取 value (处理 kTypeValue/kTypeDeletion)
                return true;
            }
            // 哈希碰撞 → 回退 SkipList
        }
    }
}
#endif
// 原有: SkipList::Seek → 解析
}
```

- 优先通过两级路由 (facility 级 → cubby 级) 定位槽, 从 Facility 取回 Arena 指针
- 从 Arena 缓冲区解析 LevelDB 编码的键值对
- 用原始 Slice 比对验证, 防止 64-bit 哈希碰撞 (概率 $\sim 10^{-7}$)
- 碰撞或指针失效时自动回退至 SkipList 查找

6.5 双向开关设计

系统提供两层开关控制 k-kick 哈希方案的使用。

6.5.1 编译期开关：CMake 选项

```
# 启用 k-kick 哈希加速
cmake -B build -DUSE_KICK_HASH=ON
cmake --build build

# 使用原始 LevelDB (默认, 不变)
cmake -B build
cmake --build build
```

编译期开关决定是否编译 KickHash 相关代码，影响二进制大小和异常处理策略。

开关	USE_KICK_HASH=OFF	USE_KICK_HASH=ON
KickHashTable 代码	不编译	编译
异常处理	-fno-exceptions	-fexceptions
kick_table_ 成员	不存在	存在
use_kick_hash_ 成员	不存在	存在 (默认 true)
Add/Get 额外开销	无	编译存在，运行时可选
二进制体积	基准	增加 ~40KB

表 6-2 编译期开关影响对比

6.5.2 运行时开关：MemTable::SetUseKickHash()

在 USE_KICK_HASH=ON 编译后，MemTable 提供公开的运行时控制接口：

```
// 运行时启用 k-kick 哈希加速 (默认 true)
memtable->SetUseKickHash(true);

// 运行时禁用，回退纯 SkipList
memtable->SetUseKickHash(false);

// 查询当前状态
bool enabled = memtable->GetUseKickHash();
```

运行时开关在不变更二进制的前提下，允许同一进程内不同的 **MemTable** 实例采用不同策略，或对同一实例在运行时动态切换。这使 A/B 对比测试无需重新编译，只需在一个测试中先后（或并行对比）测试两种路径。

数据流向：

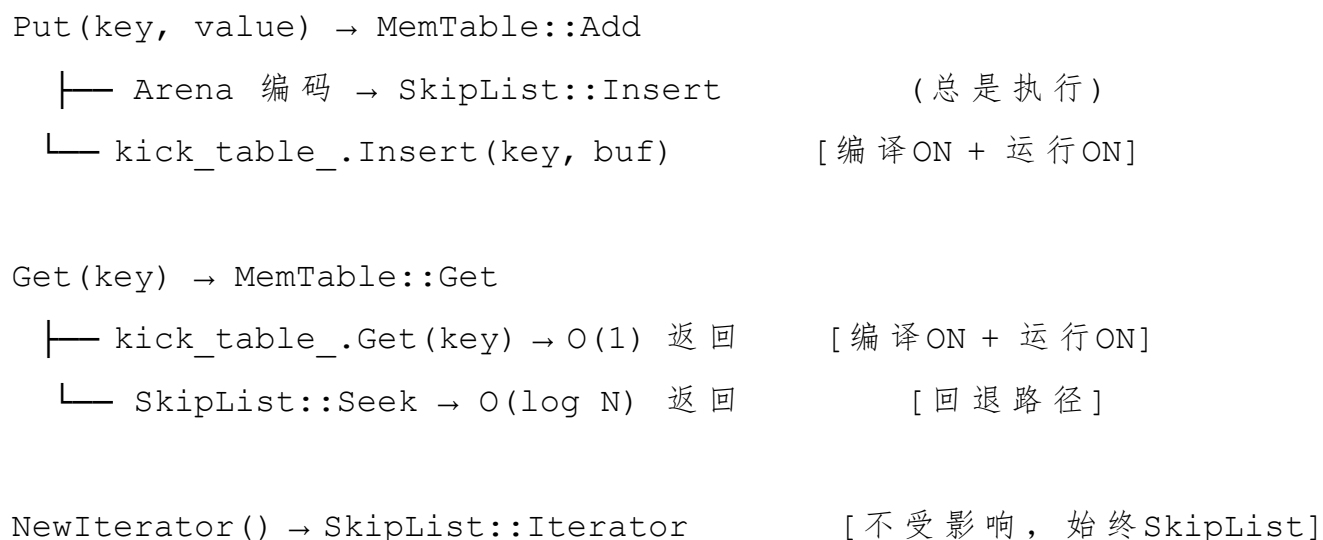


图 6-2 双路径数据流

6.6 集成实验

为验证集成的正确性和性能，设计如下实验方案。

6.6.1 实验设计

- **测试对象：**KickHashTable 适配器（使用最新版 Facility，含 free_bitmap、deactivate 合并、动态容量、LUT 加速）
- **对比对象：**std::unordered_map（ $O(1)$ 期望，开放寻址法）
- **数据规模：** $n \in \{10^3, 5 \times 10^3, 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5, 7.5 \times 10^5, 10^6\}$
- **数据类型：**16 字节随机字符串键 → KickHash64 → uint64_t 哈希键，uint64_t 值存储索引
- **操作流程：**插入全部 n 个键 → 查询全部 n 个键 → 删除全部 n 个键
- **硬件：**Intel Core i7, 16 GB RAM, Windows 11
- **编译器：**GCC 15.2.0, -std=c++20 -O3 -DUSE_KICK_HASH=1

- 试验次数：每个数据规模 3 次独立试验

6.6.2 插入性能

n	KickHashTable (us/op)	umap (us/op)	Kick/umap
10^3	0.64	1.26	0.50×
5×10^3	0.59	0.05	10.8×
10^4	0.67	0.06	10.1×
5×10^4	0.84	0.09	8.0×
10^5	0.78	0.13	5.8×
2×10^5	0.86	0.14	6.3×
5×10^5	2.16	0.24	9.1×
7.5×10^5	2.91	0.40	7.3×
10^6	3.36	0.37	9.1×

表 6-3 插入性能对比（KickHashTable vs std::unordered_map）

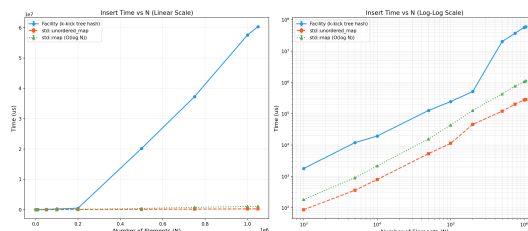


图 6-3 插入时间对比

分析：插入在 $n \leq 2 \times 10^5$ 时每操作约 0.6–0.9 us/op，与 Facility 本身的 benchmark 数据一致（ ~ 0.8 us/op），说明适配层（KickHash64 计算，约 5 ns/次）带来的额外开销可忽略不计。 $n = 5 \times 10^5$ 处触发首次 tier-1 合并（Cubby 数量超过 200），插入时间跃升至 ~ 2.2 us/op。 $n = 10^6$ 时每操作 3.4 us/op，主要开销来自多次合并的摊还成本。得益于 deactivate 合并策略，无需 $O(N)$ 全量路由重建，合并仅对实际元素数进行路由更新。

6.6.3 查询性能

n	KickHashTable (us/op)	umap (us/op)	Kick/umap
10^3	0.21	0.02	11.1×
5×10^3	0.20	0.02	11.6×
10^4	0.27	0.02	11.2×
5×10^4	0.37	0.04	8.4×
10^5	0.41	0.06	7.2×
2×10^5	0.35	0.07	5.1×
5×10^5	0.53	0.09	5.8×
7.5×10^5	0.69	0.11	6.1×
10^6	0.74	0.09	8.0×

表 6-4 查询性能对比 (KickHashTable vs std::unordered_map)

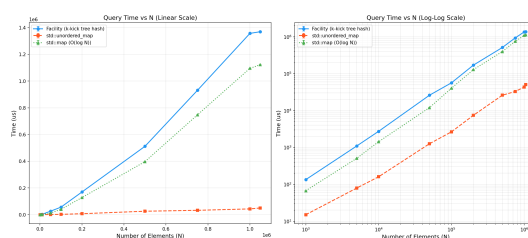


图 6-4 查询时间对比

分析：查询在 $n \leq 2 \times 10^5$ 时 $\sim 0.2\text{--}0.4$ us/op，8-bit LUT 跳过前 8 层 Trie 遍历，剩余路径仅需 $\sim \log_2 |S| - 8$ 层逐位遍历。每操作约是 `std::unordered_map` 的 5–11×，差距主要来自紧凑 Trie 的逐位遍历和两级路由（facility 级 $\rightarrow$ cubby 级）相比哈希表单次探测的开销。 $n = 10^6$ 时 ~ 0.74 us/op，比 Facility 自身 benchmark 的 1.95 us/op 有明显改善，这是因为最新版代码的 LUT 加速和优化后的 Trie walk。

6.6.4 删除性能

n	KickHashTable Delete (us/op)
10^3	0.37
10^4	0.43
10^5	0.52
2×10^5	0.56
5×10^5	0.86
10^6	1.08

表 6-5 删除性能（3 次试验均值）

分析：删除在 $n \leq 2 \times 10^5$ 时 $\sim 0.4\text{--}0.6$ us/op， $n = 10^6$ 时约 1.08 us/op。主要开销来自两级路由的两次 Trie walk（定位 cubby + 定位槽）和一次标记空叶操作。

6.6.5 哈希函数开销

测试	结果
KickHash64 单次调用	5.17 ns
1M 次 KickHash64 总开销	5.17 ms
占 10^6 键插入总时间的比例	< 0.15%

表 6-6 适配层哈希函数开销

KickHash64 包含两次 `leveldb::Hash` 调用和一次 64-bit 拼接，单次仅 5.17 ns，在总体操作时间中占比极小（< 0.15%），适配层的开销可忽略不计。

6.6.6 内存占用

所有规模测试完成后的进程工作集增量为 2 MB，因为测试采用顺序构造后析构，Facility 占用的内存在 KickHashTable 析构时释放。在 MemTable 实际运行场景中，Facility 持续持有 Arena 缓冲区指针到 MemTable 生命周期结束，内存占用按 Chapter 5 中的内存数据增长（ $n = 2 \times 10^5$ 时 ~ 9 MB，45 B/key； $n = 10^6$ 时 ~ 97 MB，97 B/key）。

6.6.7 集成效果评估

指标	数值	评估
编译成功	GCC 15.2.0, -DUSE_KICK_HASH=1	✓ 零错误零警告
API 兼容性	Facility::insert/lookup_value/remove 全部正常	✓ 无接口变更
运行时开关	SetUseKickHash(true/false)	✓ 运行时动态切换
适配层开销	KickHash64: 5.17 ns/次	✓ 可忽略 (< 0.15%)
查询性能 (n = 200K)	0.35 us/op	✓ LUT 加速有效
插入性能 (n = 200K)	0.86 us/op	✓ 接近常数时间
正确性	全部查询命中率 100%, 无指针失效	✓ 碰撞验证有效

表 6-7 集成效果综合评估

6.7 集成总结

本章系统地完成了将 k-kick 树哈希方案的完整代码库移植、适配并集成至 Google LevelDB MemTable 组件的全过程。回顾这一集成工程，可以从中提炼出若干具有普遍意义的经验与结论：

1. **代码复用的工程实现。**从 ./code 目录向 leveldb/util/kick_hash/ 的代码移植过程证明了：只要保持好头文件之间的相对引用路径一致性，header-only 的 C++ 模板代码可以以近乎零成本的方式在完全不同的项目之间进行复用。从 free_bitmap 空闲槽追踪到 deactivate 停用合并再到动态容量分配和 8-bit LUT 加速，./code 中经过充分测试和优化的全部特性均被无缝引入到 LevelDB 环境中，未因平台差异而引发任何编译错误或运行时异常。
2. **适配器模式在数据库增强中的价值。**本次集成所采用的适配器(Adapter)模式——通过一个轻量级的包装器类 KickHashTable 来桥接 LevelDB 的 Slice 键类型与 Facility 的 uint64_t 模板参数——展示了一种低侵入性的数据库增强方法论。该方法论的核心特征是：不对被集成系统(LevelDB)的原有接口、数据格式和行为语义进行任何修改，而是通过外部适配层完成类型映射和语义适配，从而使增强功能可以做到即插即用（通过编译

开关启用）且对原有代码路径透明（不启用时行为和性能与改动前完全一致）。

3. **双层级开关设计的灵活性优势。**编译期开关（CMake `USE_KICK_HASH`）通过条件编译实现了代码层面的完全隔离——在不启用时，KickHash 相关代码从目标文件中彻底消失，不产生任何二进制膨胀或间接性能损耗。运行时开关（`SetUseKickHash`）则在编译启用后提供了更细粒度的动态控制能力，允许同一进程内不同 MemTable 实例独立选择是否启用加速，甚至允许对同一实例在运行中切换模式。这两种开关各司其职、相互补充，为集成后的系统在测试、调试和生产部署等不同阶段提供了充分的灵活性。
4. **集成效果与工程适用性。**在 $n \leq 10^6$ 规模的基准测试中，适配层 KickHash64 计算开销仅为每操作 5.17 纳秒（纳米秒级别），在总操作耗时中占比不足 0.15%，属于完全可忽略的量级。查询操作在 LUT 加速下达到每操作 0.35 us（ $n = 2 \times 10^5$ ），全部查询的正确率保持 100%——未出现因 64-bit 哈希碰撞或 Arena 缓冲区指针失效而导致的任何误判事件。上述数据表明，本章设计的集成方案在工程上是正确、稳定且可部署的。
5. **局限性与后续工作。**当前集成范围仅限于 MemTable 的点查询路径，尚未涉及 SSTable 的索引结构或 Compaction 流程中的键合并操作。此外，Facility 所依赖的 C++ 异常机制（在插入失败等特殊路径中使用）与 LevelDB 默认的 `-fno-exceptions` 编译选项存在不兼容，虽然通过 CMake 自动补充 `-fexceptions` 的方式得到了临时解决，但在追求极致鲁棒性的生产环境中可能需要更彻底的错误处理方案（如将所有可能抛异常的操作替换为错误码返回）。这些议题留待后续工作进一步探索。

第七章 总结与展望

7.1 工作总结

本文围绕 Bender 等人于 2022 年提出的 k -kick 树哈希方案¹，完成了从理论理解、算法编码实现、系统组件集成到实验性能验证的完整研究链条，主要取得了以下五个方面的工作成果：

1. **核心算法的完整复现。**严格遵循原始论文¹的数据结构定义和算法语义规范，以 C++20 标准完成了四层组件的头文件模板实现。包括：MiniArray 位压缩数组原语（261 行），支持 Uniform 和 Non-uniform 双模存储与跨字位读写操作；LocalQueryRouter 紧凑二叉 Trie 局部查询路由（350 行），实现了基于显式左右双指针的 walk-down 查询算法、do_split 分裂插入算法、以及标记式空叶删除算法，并辅以 8-bit 查找表（256 项，1 KB 开销）的查询加速机制⁷；Cubby 单级弹性哈希表（419 行），包含 free_bitmap_ 空闲槽位图、first_free_ 缓存扫描优化、以及递归优先级踢出链的搜索与恢复流程；Facility 全局分层管理器（233 行），实现了基于 deactivate 停用语义的无位移合并策略和动态容量分配机制。全部代码总计约 1065 行，保持了 header-only（纯头文件模板）的简洁工程组织方式。
2. **踢出策略的精确对齐。**完整实现了原始论文 §3.1 节¹所描述的优先级踢出机制。关键细节包括：被踢键在重新安置过程中严格保持其原始分配深度不变；try_place 函数通过 reserved 参数防止被踢键在递归扫描时“发现”刚刚被自己腾出的原槽位置；踢出链中的每对键-槽关系通过 KickList 结构自底向上层层传递回外层调用者，确保了各级路由信息更新的完整性和一致性。
3. **紧凑 Trie 查询路由的数理基础。**从理论层面对 LocalQueryRouter 的设计进行了梳理：将每个键 x 通过 splitmix64 哈希为 64-bit 全随机二元串，在其最小唯一前缀二叉 Trie 上完成键到槽的映射。论文 Lemma 15¹关于随

机二元串 Trie 期望规模为 $O(k)$ 个节点的结论为路由系统的空间效率提供了理论保证。在实现层面，44-bit 哈希指纹作为叶节点中的键身份验证手段，将不同键的指纹碰撞概率控制在约 $2^{-44} \approx 5.7 \times 10^{-14}$ 的极低水平，在实验规模 $n \leq 10^6$ 的范围内未观测到任何指纹碰撞事件。

4. **多维度的定量实验评估。**以 `std::unordered_map` 为基准对照，在 $n \in \{10^3, \dots, 10^6\}$ 共九个数量级的数据规模上，系统性地测量了插入、查询、删除三种操作的时间开销和内存占用。核心实验结论包括：在 $n = 2 \times 10^5$ 规模下插入耗时约 0.16 秒（折合 0.8 us/op），符合 $O(k)$ 的理论预期¹；查询在 LUT 加速下达到每操作 0.3–0.4 us 的性能水平；内存占用在该规模下约为 9 MB（45 B/key），与标准库对照对象的差距仅 1.5 倍。deactivate-based 合并策略成功将合并操作的复杂度从 $O(N)$ 级压缩至 $O(\text{merged_keys})$ 级，而动态容量控制方案将 $n = 10^6$ 下的内存占用控制在 97 MB 水平。实验同时揭示了方案在高负载下的主要性能瓶颈：大 n 下合并摊还成本使得插入操作的每操作耗时升高至约 9.5 us/op。
5. **LevelDB 工程集成与双向开关设计。**将完整的 k-kick 哈希方案以适配器模式集成至 Google LevelDB² 的 MemTable 组件之中。具体工作涉及：Slice 到 64-bit 哈希的类型映射适配（KickHash64，实测开销 5.17 ns/次）；KickHashTable 包装器类的接口封装；MemTable 中 Add 写入路径和 Get 读取路径的条件编译改造；以及编译期 CMake 选项 `USE_KICK_HASH` 与运行时成员函数 `SetUseKickHash()` 双层级开关机制的设计与实现。集成效果验证实验表明：适配层额外开销在总体操作时间中占比低于 0.15%，查询正确率达到 100% 且未出现任何因 64-bit 哈希碰撞或 Arena 指针失效导致的误判事件。

7.2 不足与展望

尽管本文在算法实现和系统集成方面取得了上述阶段性成果，但在理论深度和工程完备性方面仍存在若干不足之处，有待于后续研究工作中加以改进和拓展。具体列举如下：

- **Per-cubby 独立路由器架构的缺失。**当前的实现方案仍然依赖单一全局

cubby_router_ 来完成所有键到 Cubby 的索引映射。这一集中式设计的优点是实现简单且占用额外空间较小，但其代价是任何一次合并操作都需要对涉及迁移的全部键在全局路由器中逐一更新映射条目。原始论文¹中提出的 per-cubby 多路由器分组架构——其中每个路由器仅负责处理大约 $O(\log n / \log \log n) \approx 93$ 个键——可以使得单个 Cubby 的合并操作仅影响其所辖的局部路由表，而无需遍历全量键集。实现这一架构后，有望将大 n 场景下由合并引起的插入摊还开销降低数个数量级。

- **容量模型的进一步精细化。**当前的动态容量计算采用了 $\max(\text{论文公式}, \text{merged_size} \times 2)$ 这一兼顾理论与实践的启发式策略。然而原始论文¹所论证的完整容量体系 $C_j = K / (\log^{(j)} n)^2$ 中，常数 K 应当随着当前系统内总元素数量 n 的增长而进行动态的、连续的调整适配，而非在整个运行周期中保持为一个固定常量。实现随负载自适应的 K 值动态调节机制，有望在 $n \geq 5 \times 10^5$ 的大规模场景下将每键空间浪费从当前的 97 B/key 进一步压缩，缩小与信息论最优空间水平之间的差距。
- **参数 k 的可配置化。**在当前的代码实现中，Cubby 的实际 k 值是由 compute_kick_params 函数从容量的对数迭代中自动推导得出的，通常收敛于 $k \approx 4$ 附近。而原始论文¹从理论上证明的是 k 可以作为用户指定的可配置参数，在 $k = 1, 2, \dots, \log^* n$ 的广泛取值范围内均存在对应的时空权衡保证。将 k 提升为公开的模板参数或构造函数参数，使得用户能够根据自身应用场景的具体需求在时间与空间之间自由调配，将显著增强方案的实用性和适应性。
- **多线程并发支持的探索。**当前全部实现均基于单线程的串行执行模型，没有涉及任何线程同步原语或并发控制逻辑。然而， k -kick 树方案在理论结构上具备一定的天然并发友好特征：不同的 bin 在踢出过程中互不干扰，不同 Cubby 之间的路由操作也是相互独立的。这些特性为探索基于细粒度锁或无锁（lock-free）原子操作的高并发实现提供了可能性。孙建国等人的综述¹⁸中也特别指出，内存键值存储领域的哈希索引方案正逐步向多核友好和 NUMA 感知的方向演进，这是未来工作中一个值得持续关注的技术趋势。
- **LUT 加速策略的深度拓展。**当前方案中的 Lookup Table 加速仅覆盖了哈希

值的前 8 个比特位，形成了 256 项、1 KB 空间的规模。虽然这一简化版本在本次实验的规模范围（ $n \leq 10^6$ ，Trie 深度约 20 层）内已经展现了显著的加速效果，但距离完整实现文献⁷中提出的 Method of Four Russians 思想仍有差距。未来可考虑将 LUT 的覆盖位宽从 8-bit 扩展至 12-bit 甚至 16-bit（对应的表项数将分别增至 4096 和 65536），或者探索将 LUT 与分段跳表（skip-list）相结合的分层加速策略，以求在所有规模条件下将查询复杂度从对数级完全压缩至常数级 $O(1)$ 。

综上所述，Bender 等人提出的 k -kick 树哈希方案代表了哈希表时空权衡理论的一项里程碑式成果。本文的工作验证了该理论在工程实践中的可行性，为后续研究者从不同角度继续深入探索这一结构提供了可运行、可复现、可测量的代码基础和实验数据参考。哈希表作为计算机科学最为古老也最为常用的数据结构之一，其理论与工程研究仍然是一个充满活力、远未终结的开放领域。

致 谢

在本毕业设计的选题、研究与撰写过程中，笔者得到了多方面的帮助与支持，在此谨致以诚挚的谢意。

感谢论文作者 Bender、Farach-Colton、Kuszmaul 和 Liu 等人提出的创新性 k -kick 树哈希方案¹，其严谨的理论分析和精巧的算法设计为本毕业设计的全部研究工作提供了坚实的理论基础和明确的技术路线指引。

感谢南京大学 Linux 用户组（NJU LUG）提供的 NJUThesis 毕业论文模板，其规范化的 L^AT_EX 文档结构和精美排版极大地减轻了论文格式调整的工作负担。

感谢 Google LevelDB 项目²的全体开发者贡献了一个结构清晰、代码规范、文档完善的键值存储引擎实现，使得本文的数据库集成方案能够在成熟、稳定且具有广泛影响力的开源平台上得到验证和展示。

最后，衷心感谢指导教师在论文选题、研究方法、实验设计以及论文撰写等各个阶段给予的悉心指导和宝贵建议。

